Rapport de d’analyse de l’utilisation d’une base SQL vs NoSQL

SAÉ 5.02

Migration d'une Base de Données Relationnelle vers un Modèle Graphe pour l’Analyse des Crimes et Délits (2012-2021)

Yanis Thollet

Ugo Miranda

Thibaud Muller

BUT3

Année 2025-2026

Table des matières

SAÉ 5.02 1
1.Introduction 2
2.Extraction et Nettoyage des données par Python 2
3.Migration vers une Base de Données Relationnelle 3
4.Modélisation de la base de données non relationnelle 4
5.Test de validations entre SQL et Neo4j 6
6.Benchmark requêtes 6
7. Résultats et comparaisons 11
8.Liens GitHub et vidéo YouTube 11

1.Introduction

Cette SAE avait pour but l'analyse des crimes et délits réalisés pour le Ministère de l'Intérieur (2012-2021). L'objectif principal est de partir d'un jeu de données avec plusieurs fichiers sources au format CSV, de migrer les données dans un système relationnel classique (PostgreSQL) puis vers une base de données orientée graphe (Neo4j). La finalité est de confronter le modèle relationnel (PostgreSQL) et le modèle orienté graphe (Neo4j) sur différentes requêtes métiers de complexité différentes et d'établir quel modèle semble le plus adapté. L'objectif n'est pas de substituer intégralement une technologie à une autre, mais d'identifier les limites du SQL face à l'analyse spatiale

2.Extraction et Nettoyage des données par Python

A l'aide d'un script Python et de librairies comme pandas, nous avons pu extraire les données des différents fichiers CSV à savoir data.xlsx contenant la base de données sur les crimes, et communes_adjacentes_2022.csv qui nous permet de connaître le voisinage des communes entre elles. Il a donc fallu dans un premier temps nettoyer ces données brutes afin d'uniformiser les données/formats/notations et les rendre exploitables en supprimant les accents, les espaces, la ponctuation pour ne garder que les lettres de A à Z et les chiffres. Nous avons ensuite pu joindre ces deux fichiers de données et obtenir une base de données propre et exploitable nous permettant une intégration SQL puis NoSQL de qualité.

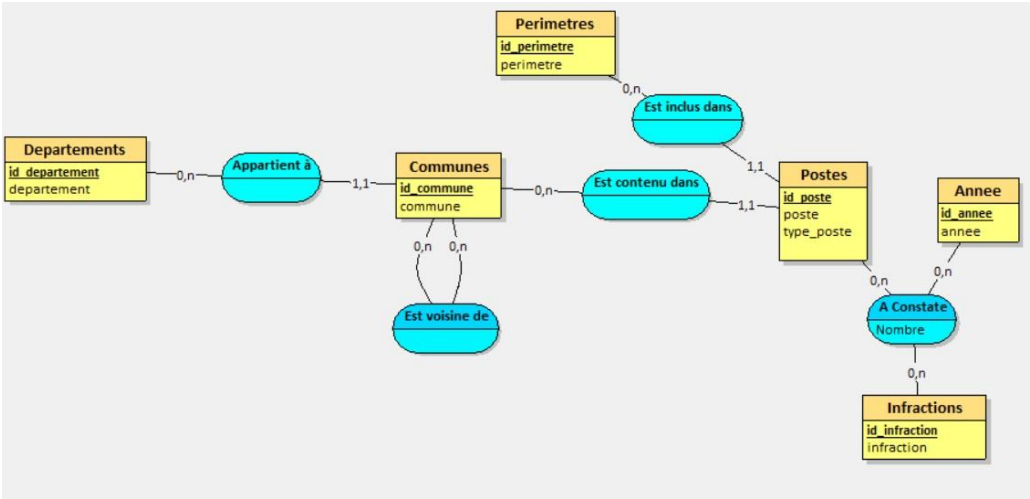
3.Migration vers une Base de Données Relationnelle

Le modèle que nous avons choisi s'articule autour de l'association centrale « A Constate » (qui porte la propriété Nombre pour quantifier les incidents) reliant les entités Annee (qui répertorie les années d'exercice), Infractions (qui référence les différents types de crimes et délits) et Postes (le lieu d'enregistrement). L'entité Postes permet ensuite de récupérer l'information géographique et administrative : elle est reliée aux Perimetres via l'association « Est inclus dans » (définissant les zones d'intervention) et aux Communes via l'association « Est contenu dans » (pour la localisation). Enfin, ce maillage territorial est complété par le lien entre les Communes et leurs Departements (« Appartient à »), ainsi que par l'auto-association « Est voisine de » qui permet de relier directement les communes limitrophes entre elles afin d'étudier la topologie.

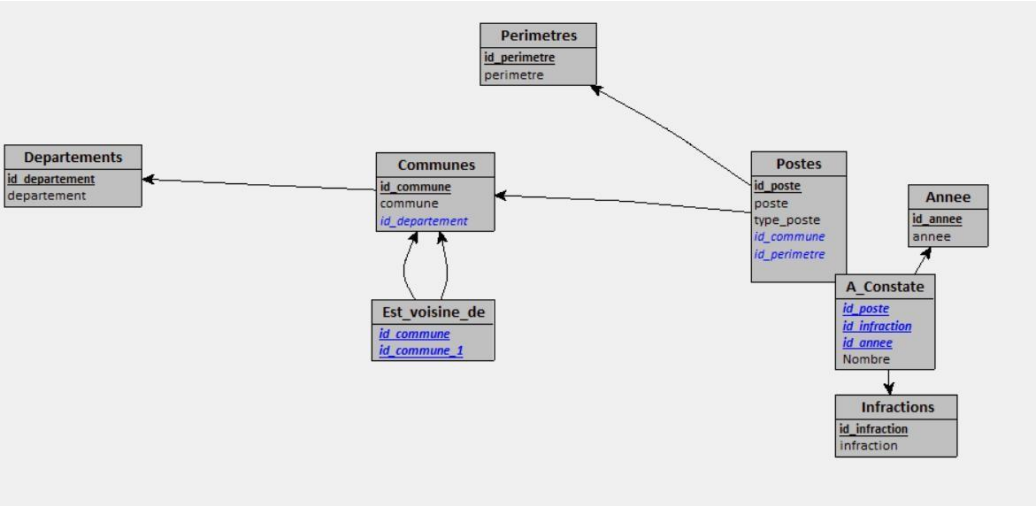
Ensuite on va créer cette base de données à l'aide de la bibliothèque sqlalchemy. On va configurer la connexion pour héberger localement notre base criminalite_db sur un port de notre machine (ici 5432). L'insertion brute est dangereuse en SQL, car la réinsertion d'entités existantes fausserait les clés primaires ou générerait des doublons. Nous avons développé une fonction Python qui détecte les lignes déjà présentes en base avant d'effectuer l'insertion. Seules les nouvelles dimensions et les nouveaux constats d'infractions sont poussés vers la base via la méthode to_sql (if_exists='append'). Si le script est relancé par erreur sur un même fichier, aucune donnée n'est dupliquée.

Enfin maintenant que la structure de notre base de données est créée on va insérer nos données. Pour ce faire, on utilisera Pandas pour transformer les DataFrames en tables SQL.

MCD :



MLD :



4.Modélisation de la base de données non relationnelle

La migration de la base Postgres vers Neo4j a aussi été faite par script Python et en suivant un modèle de graphe que nous avons trouvé parfaitement adapté au contexte de la SAE. L'objectif ici était de transformer les lignes et colonnes en nœuds et relations. Cette transformation s'est faite en traduisant les listes en dictionnaires de données pour ensuite obtenir des records.

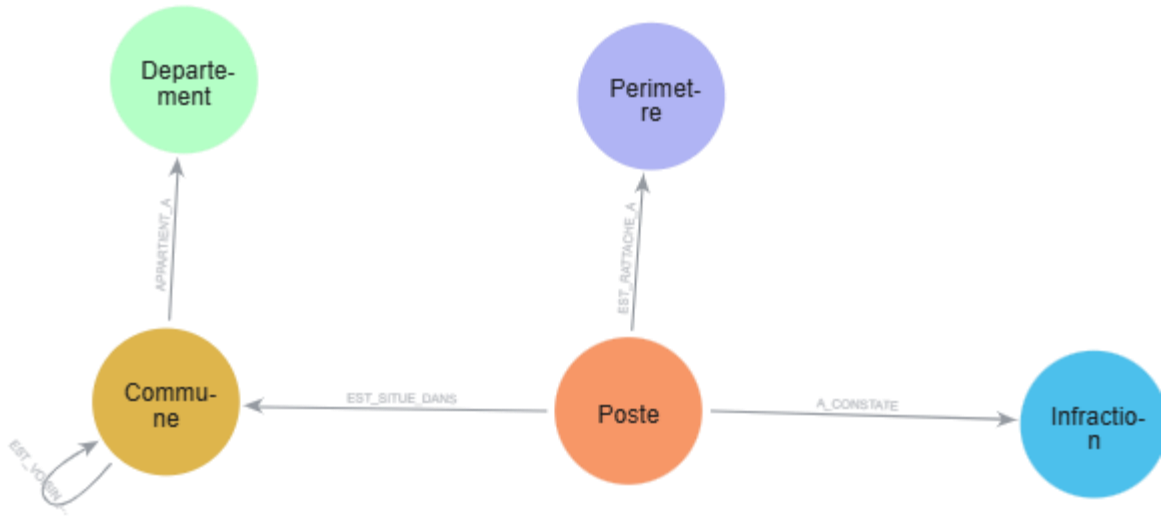
L'un des enjeux majeurs de ce projet est de garantir la durabilité du projet sur le long terme. Lorsque le Ministère publiera les statistiques d'une nouvelle année, les scripts de migration doivent pouvoir modifier/supprimer/ajouter de nouvelles données de façon cohérente, sans perte d'information et sans créer de doublons.

Dans ce cas, la mise à jour d'une base orientée graphe est structurellement plus souple grâce à la nature du langage Cypher :

Au lieu d'utiliser la commande CREATE, notre script Python utilise systématiquement la commande MERGE. Cette clause vérifie d'abord si le nœud (ou la relation) existe déjà avec les propriétés spécifiées afin d'assurer une perte minime de données lors de la migration Postgres vers Neo4j.

Si le nœud existe, Neo4j ne fait rien (ou le met à jour si on utilise ON MATCH SET). S'il n'existe pas, il le crée. Ainsi, l'ajout d'une nouvelle année d'infractions va simplement attacher de nouvelles relations [:A_CONSTATE] aux nœuds Poste et Infraction existants, sans risquer de dupliquer les postes de police. Enfin, pour gérer les millions de nouvelles lignes annuelles, l'insertion vers Neo4j est gérée par lots (batchs de 30 000 requêtes), ce qui évite de saturer la mémoire de la machine.

Modèle graphe utilisé :



5.Test de validations entre SQL et Neo4j

On vérifie qu'on a le même nombre de communes :

- **SQL :** SELECT COUNT(*) FROM tbl_communes;
- **Neo4j :** MATCH (c:Commune) RETURN count(c);

On obtient 1 050 dans les deux cas

On vérifie qu'on a le même nombre (distrinct) d'infractions

- **SQL :** SELECT COUNT(*) FROM tbl_a_constate;
- **Neo4j :** MATCH ()-[r:A_CONSTATE]->() RETURN count(r);

On obtient 1 953 099 dans les deux cas

On vérifie Le nombre total d'infractions :

- **SQL :** SELECT SUM(nombre) FROM tbl_a_constate;
- **Neo4j :** MATCH ()-[r:A_CONSTATE]->() RETURN sum(r.nombre);

On obtient 37 255 853 dans les deux cas (pour valider ce résultat nous avons comparé ce résultat à celui du contenu du fichier d'origine et on obtient le même résultat)

6.Benchmark requêtes

Requête 1 : Le nombre total d'incidents (Agrégation globale)

Objectif : Comparer la vitesse sur un calcul basique portant sur toute la base.

Resultat : 37 255 853

SQL : 0.338s

```
SELECT SUM(nombre)

FROM tbl_a_constate;
```

Neo4j (Cypher) : 0.580s

```
MATCH ()-[r:A_CONSTATE]->()
```

```
RETURN sum(r.nombre) AS total_incidents;
```

Requête 2 : Évolution des constats par type de force de l'ordre (Séries temporelles)

Objectif : Montrer une jointure classique.

SQL : 1.363s

```
SELECT a.annee, p.type_poste, SUM(ac.nombre) AS total_crimes
FROM tbl_a_constate ac
JOIN tbl_annees a ON ac.id_annee = a.id_annee
JOIN tbl_postes p ON ac.id_poste = p.id_poste
GROUP BY a.annee, p.type_poste
ORDER BY a.annee, p.type_poste;
```

Neo4j (Cypher) : 1.965s

```
MATCH (p:Poste)-[r:A_CONSTATE]->(i:Infraction)
RETURN r.annee AS annee, p.type AS type_poste, sum(r.nombre) AS total_crimes
ORDER BY annee, type_poste;
```

Requête 3 : Top 3 des crimes par département

Objectif : effectuer des jointures

SQL : 0.861s

```
WITH RankedInfractions AS (
  SELECT
    d.departement AS dep,
    i.infraction,
    SUM(ac.nombre) AS Total,
    ROW_NUMBER() OVER(PARTITION BY d.departement ORDER BY SUM(ac.nombre) DESC) AS rang
  FROM tbl_a_constate ac
```

```
    JOIN tbl_postes p ON ac.id_poste = p.id_poste
    JOIN tbl_communes c ON p.id_commune = c.id_commune
    JOIN tbl_departements d ON c.id_departement = d.id_departement
    JOIN tbl_infractions i ON ac.id_infraction = i.id_infraction
    GROUP BY d.departement, i.infraction
)
SELECT dep, infraction, Total
FROM RankedInfractions
WHERE rang <= 3;
```

Neo4j (Cypher) : 2.594s

```
MATCH (d:Departement)<-[:APPARTIENT_A]-(c:Commune)<-[:EST_SITUE_DANS]-(p:Poste)-[r:A_CONSTATE]->(i:Infraction)
WITH d.libelle AS Departement, i.libelle AS Infraction, sum(r.nombre) AS Total_Crimes
ORDER BY Total_Crimes DESC
WITH Departement, collect({infraction: Infraction, total: Total_Crimes})[0..2] AS Top3_Infractions
RETURN Departement, Top3_Infractions
ORDER BY Departement;
```

Requête 4 : Afficher les 10 infractions les plus constatées à Lyon ET dans ses communes limitrophes

SQL : 0.219s

C'est très verbeux en SQL car il faut faire des UNION et traverser la table de jointure tbl_communes_voisines.

```
WITH CommunesCibles AS (
    -- On prend Lyon
    SELECT id_commune FROM tbl_communes WHERE commune = 'LYON'
    UNION
    -- Et on ajoute les voisins directs de Lyon
    SELECT cv.id_commune_voisine
    FROM tbl_communes_voisines cv
    JOIN tbl_communes c ON cv.id_commune = c.id_commune
    WHERE c.commune = 'LYON'
)
SELECT i.infraction, SUM(ac.nombre) as total
FROM tbl_a_constate ac
JOIN tbl_postes p ON ac.id_poste = p.id_poste
JOIN CommunesCibles cc ON p.id_commune = cc.id_commune
JOIN tbl_infractions i ON ac.id_infraction = i.id_infraction
GROUP BY i.infraction
ORDER BY total DESC
LIMIT 10;
```

Neo4j (Cypher) : 0.269s.

```
MATCH (c1:Commune {libelle: 'LYON'})-[:EST_VOISIN_DE*0..1]-(c2:Commune)
MATCH (c2)<-[:EST_SITUE_DANS]-(p:Poste)-[r:A_CONSTATE]->(i:Infraction)
RETURN i.libelle AS infraction, sum(r.nombre) AS total
ORDER BY total DESC
LIMIT 10;
```

Requête 5 : Le cas d'usage "Presque impossible" en SQL

Objectif : Identifier les zones à risque de propagation. On cherche les communes situées à 2 sauts (voisins de voisins) d'une commune ayant eu plus de 1000 cambriolages en 2021.

SQL :

cela demanderait des auto-jointures multiples sur tbl_communes_ qui seraient illisibles et s'exécuteraient très lentement.

Neo4j (Cypher) : 0.147s

```
MATCH (p1:Poste)-[r:A_CONSTATE {annee: 2021}]->(i:Infraction)

WHERE i.libelle CONTAINS 'Cambriolages' AND r.nombre > 1000

MATCH (p1)-[:EST_SITUE_DANS]->(c_source:Commune)-[:EST_VOISIN_DE*1..2](c_voisine:Commune)

RETURN DISTINCT c_voisine.libelle AS Commune_Vulnerable

ORDER BY Commune_Vulnerable;
```

"""

Requete	Temps SQL	Temps Neo4j
Le nombre total d'incidents	0.338s	0.580s
Évolution des constats par type de force de l'ordre	1.363s	1.965s
Top 3 des crimes par département	0.861s	2.594s
Afficher les 10 infractions les plus constatées à Lyon ET dans ses communes limitrophes	0.219s	0.269s
les communes situées à 2 sauts (voisins de voisins) d'une commune ayant eu plus de 1000 cambriolages en 2021.	-	0.147s

7. Résultats et comparaisons

Les forces du modèle Relationnel

Le modèle relationnel reste extrêmement performant pour les opérations de type "Business Intelligence" classiques (agrégations globales, séries temporelles, fenêtrage).

Agrégations massives : Sur le calcul du nombre total d'incidents (37 255 853 lignes), PostgreSQL est plus rapide (0.338s contre 0.580s pour Neo4j).

Tris et Groupements complexes : Pour identifier le Top 3 des crimes par département , l'utilisation des fonctions de fenêtrage SQL (ROW_NUMBER() OVER) reste plus optimisée (0.861s) que le traitement Cypher équivalent qui nécessite un regroupement et un découpage de listes (collect()[0..2]) plus coûteux (2.594s).

Les limites du SQL face aux données interconnectées

Les faiblesses du modèle relationnel apparaissent dès lors que l'on interroge la dimension géographique et les relations entre entités (la table de liaison Est_voisine_de).

Pour interroger une commune et ses voisines directes (Requête 4), le code SQL requiert des opérateurs UNION lourds pour traverser la table de jointure. À l'inverse, Cypher nécessite uniquement d'utiliser l'opérateur *0..1.

La limite structurelle du SQL est atteinte lors de la "Requête 5". Vouloir identifier les zones à risque de propagation à 2 sauts (voisins de voisins d'une commune très ciblée par les cambriolages en 2021) nécessite en SQL des auto-jointures multiples sur la table tbl_communes_voisines. Cette approche génère un produit cartésien très volumineux, rendant la requête illisible et ses temps d'exécution trop longs pour être mis en production.

La valeur ajoutée du Modèle Graphe (Neo4j)

C'est sur ces analyses de "propagation" que la base de données graphe justifie son adoption.

Grâce au stockage natif des relations, Neo4j n'a pas besoin de calculer des jointures à l'exécution.

Sur la Requête 5 (Voisins de voisins), Neo4j traverse la relation [:EST_VOISIN_DE*1..2] et retourne le résultat en seulement **0.147s**

8.Liens GitHub et vidéo YouTube

Gitbhub : https://github.com/ythollet/SAE-crimes/tree/main

Youtube : https://youtu.be/PNJS0-Gona0